



⋮ Btrfs

[🌐 8 languages](#) ▾[Page](#) [Discussion](#)[Read](#)[View source](#)[View history](#)[Tools](#) ▾Related articles: [File systems](#) | [Snapper](#) | [Timeshift](#) | [Yabsnap](#)From the **Btrfs Documentation** [🔗](#):

Btrfs is a modern copy on write (COW) file system for Linux aimed at implementing advanced features while also focusing on fault tolerance, repair and easy administration.

Note

Like some other file systems, Btrfs is under continuous development. This means that specific features may not be ready for everyday use yet. To see if your use case might be affected, check the Btrfs Documentation's **Status** [🔗](#) and the **#Known issues** section of this article.

1 Preparation

For user space utilities, **install** the **btrfs-progs** package that is required for basic operations.

If you need to boot from a Btrfs file system (i.e., your kernel and initramfs reside on a Btrfs partition), check if your **boot loader** supports Btrfs.

2 File system creation

The following shows how to create a new Btrfs **file system**. To convert an Ext3/4 partition to Btrfs, see **#Ext3/4 to Btrfs conversion**. To use a partitionless setup, see **#Partitionless Btrfs disk**.

See `mkfs.btrfs(8)` for more information.

2.1 File system on a single device

To create a Btrfs file system on partition `/dev/partition` :

```
# mkfs.btrfs -L mylabel /dev/partition
```

The Btrfs default nodesize for metadata is 16 KiB, while the default sectorsize for data is equal to page size and autodetected. To use a larger nodesize for metadata (must be a multiple of sectorsize, up to 64 KiB is allowed),

specify a value for the `nodesize` via the `-n` switch as shown in this example using 32 KiB blocks:

```
# mkfs.btrfs -L mylabel -n 32k /dev/partition
```

Note

According to `mkfs.btrfs(8)` § **OPTIONS**, "[a] smaller node size increases fragmentation but leads to taller b-trees which in turn leads to lower locking contention. Higher node sizes give better packing and less fragmentation at the cost of more expensive memory operations while updating the metadata blocks".

2.2 Multi-device file system

Warning

- The RAID 5 and RAID 6 modes of Btrfs are fatally flawed, and should not be used for "anything but testing with throw-away data." **List of known problems and partial workarounds** [↗](#). See `btrfs(5)` § **RAID56 STATUS AND RECOMMENDED PRACTICES** for status updates.
- By default, **systemd** disables **CoW** for `/var/log/journal`, which can cause data corruption on RAID 1 (see **#Disabling CoW**). To prevent this, create an empty file `/etc/tmpfiles.d/journal-nocow.conf` to override `/usr/lib/tmpfiles.d/journal-nocow.conf` (see `tmpfiles.d(5)` § **CONFIGURATION DIRECTORIES AND PRECEDENCE**).

Multiple devices can be used to create a RAID. Supported RAID levels include RAID 0, RAID 1, RAID 10, RAID 5 and RAID 6. Starting from kernel 5.5 RAID1c3 and RAID1c4 for 3- and 4- copies of RAID 1 level. The RAID levels can be configured separately for data and metadata using the `-d` and `-m` options respectively. By default, the data has one copy (`single`) and the metadata is mirrored (`raid1`). This is similar to creating a **JBOD configuration** [↗](#), where disks are seen as one file system, but files are not duplicated. See **Using Btrfs with Multiple Devices** [↗](#) for more information about how to create a Btrfs RAID volume.

```
# mkfs.btrfs -d single -m raid1 /dev/part1 /dev/part2 ...
```

You must include either the `udev` hook, `systemd` hook or the `btrfs` hook in `/etc/mkinitcpio.conf` in order to use multiple Btrfs devices in a pool. See the **Mkinitcpio#Common hooks** article for more information.

Once the file system is created, it is advised to run the following command to scan for multi-device Btrfs file systems and register them, allowing to mount the multi-device file system by specifying only one member:

```
# btrfs device scan
```

Note

- It is possible to add devices to a multiple-device file system later on. See the **Btrfs wiki article** [↗](#) for more information.
- Devices can be of different sizes. However, if one drive in a RAID configuration is bigger than the others, this extra space will not be used.

- Some **boot loaders** such as **Syslinux** do not support multi-device file systems.
- Btrfs does not automatically read from the fastest device, so mixing different kinds of disks results in inconsistent performance. See [1] [↗](#) for details.

See **#RAID** for advice on maintenance specific to multi-device Btrfs file systems.

2.2.1 Profiles

Btrfs uses the concept of **profiles** [↗](#) to configure mirroring, parity, and striping. In standard **RAID** [↗](#) terminology, this is called *RAID level*. Profiles for metadata (the `-m` option of `mkfs.btrfs(8)`) and data (the `-d` option of `mkfs.btrfs(8)`) may be different for the same Btrfs file system.

Some notable profiles:

single

No mirroring, no striping, no parity, enables mapping several devices to a single file system, called `LINEAR` in **mdadm** terminology.

raid0

No mirroring, striping, no parity, enables parallel access between devices, but not limited to same-size devices like traditional **mdadm** RAID.

raid1

Mirroring, no striping, no parity, enables recovering from one drive failure.

3 Configuring the file system

3.1 Copy-on-Write (CoW)

By default, Btrfs uses **copy-on-write** [↗](#) for all files all the time. Writes do not overwrite data in place; instead, a modified copy of the block is written to a new location, and metadata is updated to point at the new location. See the **Btrfs Sysadmin Guide section** [↗](#) for implementation details, as well as advantages and disadvantages.

3.1.1 Disabling CoW

Warning

Disabling CoW in Btrfs also disables checksums. Btrfs will not be able to detect corrupted `nodatacow` files. When combined with RAID 1, power outages or other sources of corruption can cause the data to become out of sync.

To disable copy-on-write for newly created files in a mounted subvolume, use the `nodatacow` mount option. This will only affect newly created files. Copy-on-write will still happen for existing files. The `nodatacow` option also disables compression. See `btrfs(5)` for details.

Note

- From `btrfs(5)` § **MOUNT OPTIONS**: "within a single file system, it is not possible to mount some subvolumes with `nodatacow` and others with `datacow`. The mount option of the first mounted subvolume applies to any other subvolumes."

- Even with Copy-on-Write disabled, it may still be triggered in certain cases. See **#Cases where CoW is still triggered** below for details.

To disable copy-on-write for single files/directories, do:

```
$ chattr +C /dir/file
```

Note

From **Btrfs documentation: File attributes** [\[1\]](#): "When set on a directory, all newly created files will inherit this attribute. Due to implementation limitations, this flag can be set/unset only on empty files."

Tip

In accordance with the note above, you can use the following trick to disable copy-on-write on existing files in a directory:

```
$ mv /path/to/dir /path/to/dir_old
$ mkdir /path/to/dir
$ chattr +C /path/to/dir
$ cp -a --reflink=never /path/to/dir_old/. /path/to/dir
$ rm -rf /path/to/dir_old
```

Make sure that the data are not used during this process. For more on the use of the `--reflink=never` option, see **#Effect on copying** below.

3.1.1.1 Cases where CoW is still triggered

Copy-on-Write will *still* be triggered if a file with the NOCOW attribute (+C) has more than one reference (for example, a copy created via `cp` using **reflink** [\[1\]](#) or a snapshot made by Btrfs).



This article or section needs language, wiki syntax or style improvements. See Help:Style for reference.

Reason: 5 levels of subsections is too much. (Discuss in **Talk:Btrfs**)

3.1.1.1.1 Effect on copying

When using `cp` to copy files, the NOCOW attribute is *not* determined by the source file, but by the destination path—either inherited from the directory or determined by whether the subvolume is mounted with the `nodatacow` option. Reflink copying is possible only if both the source and destination files either have or do not have the NOCOW attribute. The actual behavior depends on the `--reflink` option, with three possible outcomes:

- With `--reflink=auto` (default): If the NOCOW attributes match, reflink is used; otherwise, a deep copy is made.
- With `--reflink` (or `--reflink=always`): If the NOCOW attributes match, reflink is used; if they differ, the

operation fails.

- With `--reflink=never` : A deep copy is always performed.

For more information on the `--reflink` option, refer to **cp(1)**.

3.1.1.1.2 Effect on snapshots

If a file has copy-on-write disabled (NOCOW) and a **snapshot** is taken, the first write to a file block after the snapshot **will be a COW operation** because the snapshot locks the old file blocks in place. However, the file will retain the NOCOW attribute and any subsequent writes to the same file block will be in-place until the next snapshot.

Frequent snapshots can reduce the effectiveness of NOCOW, as COW is still required on the first write. To avoid copy-on-write for such files altogether, put them in a separate subvolume and do not take snapshots of that subvolume.

3.2 Compression

Btrfs supports **transparent and automatic compression**. This reduces the size of files as well as significantly increases the lifespan of flash-based media by reducing write amplification. See

Fedora:Changes/BtrfsByDefault#Compression, **[2]**, and **[3]**. It can also **improve performance**, in some cases (e.g. single thread with heavy file I/O), while obviously harming performance in other cases (e.g. multi-threaded and/or CPU intensive tasks with large file I/O). Better performance is generally achieved with the fastest compress algorithms, `zstd` and `lzo`, and some **benchmarks** provide detailed comparisons.

3.2.1 Compression types

Btrfs supports the compression algorithms `zlib` (**DEFLATE** provided by `zlib`), `lzo` (**LZO**), and `zstd` (**Zstandard** implemented by `zstd`). LZO has no compression levels, whereas `zlib` and `zstd` have adjustable compression levels (`zlib`: 1-9; `zstd`: -15 to -1, 1-15, must be integers). Higher levels provide better compression but require more processing time. Changing the levels will affect CPU and I/O throughput differently, so they should be checked / benchmarked before and after changing. For more information about compression types, see **btrfs(5) § COMPRESSION**.

Warning

Systems using older kernels or **btrfs-progs** without `zstd` support (especially levels -15 to -1) may be unable to read or repair your file system if you use this option.

3.2.2 Enabling compression

3.2.2.1 For filesystems

The `compress=alg[:level]` mount option enables automatically considering compression for every write to the filesystem, where `alg` is either `zlib`, `lzo`, `zstd`, or `no` (for no compression), and the optional `level` specifies the compression level (defaults to 3 when not specified or set to 0; not applicable for `lzo`). Using this option, Btrfs will check if compressing the **first data block** of a file write shrinks it. If it does, the entire write to that file will be compressed. If it does not, the file is marked as `NOCOMPRESS`, and **both the current write and all subsequent writes** to that file will not be compressed **[4]**. This is done to prevent making the disk wait to start

writing until all of the data to be written is fully given to Btrfs and compression is attempted.

Note

- Compression mount options are shared across all mount points of the same filesystem (including different subvolumes and bind mounts) [5] [↗](#).
- If one write to a file cannot be compressed, subsequent writes will not be compressed either.



This article or section needs expansion.

Reason: Missing reference for the "empirical testing" mentioned in the following paragraph, and the percentage descriptions in the original text are not accurate. (Discuss in **Talk:Btrfs**)

The `compress-force=alg[:level]` mount option can be used instead, which makes Btrfs check **each data block of every write** and decide whether to compress them individually. Empirical testing on multiple mixed-use systems showed that using `compress-force=zstd` can significantly improve space savings (from 10% to 20%) compared to `compress=zstd`, but this causes (slightly) higher CPU usage and longer write times without purpose. However, forcing compression is against **official Btrfs guidelines** [↗](#).

Only files created or modified after adding either of the above mount options will be automatically compressed.

Tip

To enable compression when installing Arch to an empty Btrfs partition, use the `compress` option when **mounting** the file system before installation: `mount -o compress=zstd /dev/sdXY /mnt/`. During configuration, add the same `compress` mount option to the root file system in **fstab**.

3.2.2.2 For existing files

To apply compression to existing files, use the `btrfs filesystem defragment -calg` command, where *alg* is either `zlib`, `lzo` or `zstd`. To specify the compression level, use the `-L` option, otherwise the default level is used (not applicable for `lzo`). For example, to re-compress all files with `zstd` and compression level 1, run the following command:

```
# btrfs -v filesystem defragment -r -czstd -L1 /
```

Warning

Defragmenting a file which has a COW copy (either a snapshot copy or one made with `cp`) may result in two unrelated files, potentially greatly increasing disk usage.

Using the above method to compress files is not persistent; other write operations will apply the original compression settings.

The following two methods can persistently enable compression for individual files:

```
$ chattr +c file
```

```
$ btrfs property set file compression zstd
```

The first command uses the legacy interface of file attributes inherited from the ext2 filesystem and is not flexible, defaulting to `zlib` compression algorithm. The second command can specify the compression algorithm, but specifying compression levels is not yet implemented, so the default level 3 is used (not applicable for `lzo`).

3.2.3 View compression types and ratios

compsize takes a list of files or directories (or an entire Btrfs file system) and outputs the compression types used and actual compression ratios (compressed size/uncompressed size). Uncompressed size may not match the number given by other programs such as **du(1)**, because every extent is counted once and only once—even if it has multiple **reflinks** [↗](#), or if parts of it are no longer used anywhere but have not been garbage collected.

The `-x` option keeps it on a single file system, which is useful in situations like `compsize -x /` to avoid it from attempting to access non-Btrfs subdirectories and failing the entire run.

3.3 Subvolumes

"A Btrfs subvolume is not a block device (and cannot be treated as one) instead, a Btrfs subvolume can be thought of as a POSIX file namespace. This namespace can be accessed via the top-level subvolume of the file system, or it can be mounted in its own right." [\[6\]](#) [↗](#)

Each Btrfs file system has a top-level subvolume with ID 5. This subvolume cannot be removed or replaced by another subvolume. The top-level subvolume has path `/` on the file system and other subvolumes are *nested* below the top-level subvolume. However, subvolumes can be moved around in the file system and their path may change, whereas their ID cannot.

By default, the top-level subvolume is mounted when mounting the file system. Options allow to **mount a specific subvolume** instead.

A major use case for subvolumes are **snapshots**.

See the following links for more details:

- **Btrfs Documentation** [↗](#)
- **Btrfs Wiki SysadminGuide#Subvolumes** [↗](#)
- **Btrfs Wiki Trees** [↗](#)

3.3.1 Creating a subvolume

To create a subvolume, the Btrfs file system must be mounted.

```
# btrfs subvolume create /path/to/subvolume
```

Note

- You can use `--parents` to automatically create parent directories if they do not exist.
- If the top-level subvolume is not mounted and you want to create subvolumes inside it, you can temporarily mount it by specifying `subvolid=5`. See **#Mounting subvolumes**.

3.3.2 Listing subvolumes

To see a list of all subvolumes of the filesystem that *path* belongs to:

```
# btrfs subvolume list -t path
```

-t triggers a more readable table view.

3.3.3 Deleting a subvolume

To delete a subvolume:

```
# btrfs subvolume delete /path/to/subvolume
```

If the subvolume contains other subvolumes that should be deleted, add the `-R / --recursive` option. Alternatively, a subvolume can be deleted like a regular directory (`rm -r`, `rmdir`).

Warning

Make sure that a subvolume is not mounted before deleting it. Deleting a mounted subvolume can lead to filesystem inconsistencies.

3.3.4 Mounting subvolumes

Subvolumes can be mounted using the `subvol=/path/to/subvolume` or `subvolid=objectid` mount options. One can mimic traditional file system partitions by creating various subvolumes under the top level of the file system and then mounting them at the appropriate mount points.

For example, the following mounts the top-level subvolume to `/mnt/` and creates two subvolumes named `subvol_root` and `subvol_home`. When the top-level subvolume is unmounted, the new subvolumes are mounted to `/mnt/` and `/mnt/home/`:

```
# mount device /mnt/ -o subvolid=5
# btrfs subvolume create /mnt/subvol_root/
# btrfs subvolume create /mnt/subvol_home/
# umount /mnt/
# mount -o subvol=/subvol_root device /mnt/
# mount -o subvol=/subvol_home --mkdir device /mnt/home/
```

Referencing a subvolume with the `subvol` mount option requires using the path relative to the *top-level subvolume*. This can be different from the path where the top-level subvolume is mounted.

Tip

- It is preferable to mount using `subvol=/path/to/subvolume`, rather than the `subvolid`, as the `subvolid` may change when restoring **#Snapshots**, requiring a change of mount configuration.
- Changing subvolume layouts is made simpler by not using the top-level subvolume (ID=5) as `/` (which

is done by default). Instead, consider creating a subvolume to house your actual data and mounting it as `/`.

Note

From `btrfs(5)` § **MOUNT OPTIONS**: "Most mount options apply to the **whole file system**, and only the options for the first subvolume to be mounted will take effect. This is due to lack of implementation and may change in the future."

See [Snapper#Suggested filesystem layout](#), [Btrfs SysadminGuide#Managing Snapshots](#), and [SysadminGuide#Layout](#) for example file system layouts using subvolumes.

3.3.5 Mounting subvolume as root

To use a subvolume as the root mountpoint, either make it the **default subvolume**, or specify the subvolume via a **kernel parameter** using `rootflags=subvol=/path/to/subvolume`. Edit the root mountpoint in `/etc/fstab` and specify the mount option `subvol=`. Alternatively, the subvolume can be specified with its id, `rootflags=subvolid=objectid` as kernel parameter and `subvolid=objectid` as mount option in `/etc/fstab`. It is preferable to mount using `subvol=/path/to/subvolume`, rather than the subvolid, as the subvolid may change when restoring **#Snapshots**, requiring a change of mount configuration, or else the system will not boot.

3.3.6 Changing the default sub-volume

The default sub-volume is mounted if no `subvol=` mount option is provided. To change the default subvolume, do:

```
# btrfs subvolume set-default subvolume-id /
```

where *subvolume-id* can be found by **listing**.

Changing the default subvolume with `btrfs subvolume set-default` will make the top level of the file system inaccessible, except by use of the `subvol=/` or `subvolid=5` mount options [\[7\]](#).

Note

GRUB's `grub-mkconfig` script does *not* respect Btrfs default subvolume settings—if `/` is the top-level subvolume, it will never specify `subvol=/` or `subvolid=5` mount options, which may cause boot failure; if `/` is not the top-level subvolume, it will always specify the corresponding `subvol` option, making it impossible to fall back by simply setting the default subvolume ([bug report](#)). Currently this can be resolved by manually modifying the `rootflags` **kernel parameter** in `grub.cfg` (see **#Mounting subvolumes**), or by using tools to automatically create boot entries for snapshots in **GRUB** (see **#Booting into snapshots**). Additionally, if the actual location of `grub.cfg` changes, such as when the subvolume containing `/boot` changes, you need to run `grub-install` again. See [this forum thread](#).

3.4 Quota

The concept of **disk quota** [\[8\]](#) has a long tradition in the Unix world. Traditional quotas are based on file ownership, controlling space usage by limiting the total size of all files owned by users. While this approach is straightforward for file management, it has limitations in directory-level restrictions, typically requiring partitioning at installation time and lacking dynamic adjustment capabilities. Btrfs adopts a different quota design philosophy from traditional Unix systems and does not support traditional **user- or group-based** quota functionality. This is because its modern storage features (such as **#Snapshots**, **#Copy-on-Write (CoW)**, **#Deduplication**, and **#Compression**) make traditional quota space calculations extremely complex and inaccurate [\[8\]](#) [\[9\]](#). As an alternative, Btrfs uses a subvolume-based quota mechanism, currently providing two implementation approaches — traditional **#Quota groups (qgroups)** and the newer **#Simple quotas (squotas)** (introduced in kernel 6.7 released in late 2023) to achieve similar space management effects. While this increases management complexity, it provides more flexible and precise storage control capabilities, allowing directory size limitations without partitioning and enabling dynamic quota adjustments.

Note

Quota management is only possible after mounting a Btrfs filesystem (or any of its subvolumes).

3.4.1 Quota groups (qgroups)

Warning

While **Btrfs qgroups** [\[9\]](#) are considered stable for general use, enabling qgroups, especially in environments with frequent creation of large numbers of snapshots, can still lead to performance degradation, particularly during snapshot deletion. Qgroup accounting introduces additional metadata overhead and may affect system responsiveness under high-intensity snapshot workloads.

Recent kernel improvements (5.15 and later versions) have fixed many **historical issues** [\[9\]](#), but **some limitations remain** [\[9\]](#), particularly in advanced application scenarios such as deeply nested subvolumes or complex qgroup hierarchical structures. The suggestion written here since 2016 — to carefully evaluate specific use cases before enabling qgroups — still applies today. Please be sure to consult the latest **Btrfs documentation** [\[9\]](#) for current detailed information. It is worth noting that to address the performance issues of qgroups, Btrfs introduced the **#Simple quotas (squotas)** functionality, providing an alternative solution with slightly less flexibility but better performance.

Btrfs's precise quota support is implemented through qgroups above subvolumes. Qgroups are represented using *qgroupids* in the format *Level/ID*. Subvolumes have level 0, where *0/* can be omitted, and the corresponding *ID* is the subvolume ID. For example, *0/5* represents the qgroup of the top-level subvolume. Subvolume paths can also be used instead of the *0/ID* representation. Qgroups form a *tree* hierarchy, but a qgroup can have multiple parent qgroups [\[9\]](#) [\[9\]](#). Leaf qgroups have level 0, corresponding directly to subvolumes. The *ID* of all higher-level qgroups can be freely specified. Higher-level qgroups contain lower-level qgroups, and same-level groups cannot be nested, but level does not indicate nesting depth. For example, child qgroup *0/5* can be assigned to qgroup *5/100*.

3.4.1.1 Total referenced space and exclusively referenced space

Each qgroup primarily tracks two values: the amount of *total referenced space* and the amount of *exclusively referenced space*. *Total referenced space* refers to the storage space occupied by all data accessible from within the qgroup, while *exclusively referenced space* refers to the storage space occupied by data that is only

referenced by **subvolumes** belonging to the qgroup (and not referenced by **subvolumes** outside the qgroup).

3.4.1.2 Enabling/disabling qgroups

Use the following command to enable qgroups for the filesystem containing *path*:

```
# btrfs quota enable path
```

Change `enable` to `disable` to disable qgroups.

Warning

Disabling qgroups will destroy all current qgroups and remove their relationships.

3.4.1.3 Creating/destroying qgroups

Level 0 qgroups corresponding to subvolumes are always automatically created. Even if subvolumes are created before enabling quotas, the corresponding qgroups will be automatically created when quotas are enabled. Similarly, when **deleting subvolumes**, the corresponding qgroups will be deleted and automatically disassociate from higher-level qgroups that originally contained these qgroups.

Use the following command to create qgroup *qgroupid* in the filesystem containing *path*:

```
# btrfs qgroup create qgroupid path
```

Change `create` to `destroy` to destroy the qgroup. Destroying a qgroup is similar to using `rm -d` to delete a directory — if the qgroup has child qgroups, relationships must be removed first; if only parent qgroups exist, relationships will be automatically removed.

Note

Qgroups corresponding to subvolumes cannot be deleted directly, but should be automatically deleted by directly deleting the subvolume.

3.4.1.4 Assigning/removing child qgroups

Use the following command to assign *src* as the child qgroup of *dst* in the filesystem containing *path*, note that *dst* must be at a higher level than *src*:

```
# btrfs qgroup assign src dst path
```

Change `assign` to `remove` to remove this relationship.

Note

After assigning/removing child qgroups, **#Quota rescan** may be needed, which is automatically triggered in current versions, but may cause performance overhead. See the **#Quota rescan** section for details.

3.4.1.5 Limiting/unlimiting qgroups

You can limit the total referenced space or exclusively referenced space of qgroups, or limit both simultaneously. For example, use the following command to limit the **total referenced space** of *qgroupid* to no more than 1GiB in the filesystem containing *path*:

```
# btrfs qgroup limit 1G qgroupid path
```

Note

If **enabled compression**, Btrfs currently only supports limiting the space occupied by compressed data.

If *qgroupid* is omitted, it attempts to interpret *path* as a subvolume path. Add the `-e` option (after `limit`) to limit **exclusively referenced space** instead. Change *size* to `none` to remove the limit.

3.4.1.6 Listing qgroups

Use the following command to list all qgroups in the filesystem containing *path*:

```
# btrfs qgroup show path
```

Note

Since `btrfs qgroup show` uses data from disk, `sync` may be needed before displaying accurate statistics. For this purpose, you can add the `--sync` option (after `show`) to first perform a forced synchronization operation on the filesystem containing *path*.

The "Path" column in the output may display some special values with the following meanings:

- `<toplevel>` : Qgroup corresponding to the top-level subvolume.
- `<under deletion>` : The subvolume corresponding to this qgroup has been deleted (its directory has been removed), but the subvolume metadata has not been completely cleaned up.

Tip

You can use `btrfs subvolume sync path subvolId` to wait for complete deletion of subvolumes in the filesystem containing *path* (multiple subvolume IDs can be specified; if none are specified, it waits for all currently pending subvolume deletions in the filesystem to complete). However, note to unmount the subvolume first.

- `<quota space holder>` : (Only applicable to **#Simple quotas (squotas)**) The subvolume corresponding to this qgroup has been completely deleted but related statistics are still retained. This is because simple quotas only attribute extents to the subvolume that first allocated them, and while that subvolume has been completely deleted, the extents are still referenced by other subvolumes.

According to your needs, you can add some display options (after `show`). Common options include:

- `-p` : Show parent qgroups.

- `-c` : Show child qgroups.
- `-r` : Show total referenced space limits.
- `-e` : Show exclusively referenced space limits.
- `-f` : Only list qgroups corresponding to the subvolume containing *path*.
- `-F` : Only list qgroups corresponding to the subvolume containing *path* and higher-level qgroups that contain these qgroups.

For more options, please refer to **btrfs-qgroup(8) § show**.

3.4.1.7 Quota rescan

Quota rescan  reads the metadata of all extents in the filesystem and updates the statistics of each qgroup accordingly.

Btrfs qgroups can handle many complex extent sharing and unsharing scenarios (including deletion of subvolumes and child qgroups) while maintaining accurate counts of total referenced space and exclusively referenced space. However, when qgroup relationships change (manual **assignment/removal of child qgroups**), due to insufficient recorded data (only recording total referenced space and exclusively referenced space, without explicitly recording which extents belong to shared or exclusive), quota rescan across the entire filesystem is usually required. At this time, qgroups will be marked as "inconsistent", indicating a state that requires quota rescan.

The only exception in the current implementation is when a qgroup's total referenced space equals its exclusively referenced space, meaning all data is exclusive. In this case, when performing assign/remove operations on this subvolume, it only needs to add/subtract this qgroup's referenced space to/from the parent qgroup's total referenced space and exclusively referenced space simultaneously.

Starting from kernel version 4.19, manual **assignment/removal of child qgroups** automatically triggers quota rescan (when necessary). However, since quota rescan has high overhead, can only be performed once at a time, and this method cannot wait for the rescan to complete before returning, you can add the `--no-rescan` option (after `assign / remove`) to avoid automatic triggering and manually trigger it later.

Tip

After using the `--no-rescan` option for **assignment/removal of child qgroups**, if quota rescan is needed, a corresponding warning will be displayed. Warnings can also be displayed through **#Listing qgroups**.

Use the following command to manually trigger quota rescan for the filesystem containing *path*:

```
# btrfs quota rescan path
```

The command returns immediately. Add the `-w` (`--wait`) option (after `rescan`) to wait for the rescan to complete (even if already started) before returning. Use the `-W` (`--wait-norescan`) option to only wait for the currently running rescan to complete. Use the `-s` (`--status`) option to return current progress.

3.4.2 Simple quotas (squotas)

Qgroups can handle many complex extent sharing and unsharing scenarios (including deletion of subvolumes

and child qgroups) while maintaining accurate counts of total referenced space and exclusively referenced space. However, this flexibility comes at a cost: many computational operations are global in nature, meaning that when extent reference relationships change, they affect the statistics of all qgroups that reference the extent. This can lead to slower transaction commit speeds and unacceptable delays, especially when the number of snapshots increases dramatically.

To address this limitation of qgroups, Btrfs supports a second set of quota semantics: **simple quotas (squotas)** [\[9\]](#). Squotas reuse the qgroup API and hierarchical structure model but do not track shared and exclusive usage. Instead, squotas attribute all extents to the subvolume that first allocated them. By introducing a small amount of new metadata records, this allows all accounting decisions to be completed locally in operations involving the allocation or release of data blocks themselves, completely avoiding complex and time-consuming reverse reference resolution processes.

Squotas need to be enabled at the filesystem level by simply adding the `-s ( --simple )` option (after `enable`) to the **enabling qgroups** command. Note that if qgroups are already enabled, you need to first **disable qgroups** and then enable squotas, otherwise it will not take effect [\[10\]](#) [\[9\]](#). Due to its simple calculation method, quota mode does not need (and cannot) perform **#Quota rescan**. Apart from these two points, squotas operate exactly the same as qgroups, but the so-called "total referenced space" and "exclusively referenced space" displayed and limited are all values calculated through the method described above (attributing all extents to the subvolume that first allocated them). Since squotas need to record the subvolume that first allocated extents, they only take effect for extents written after enabling. If squotas are enabled on a non-empty filesystem, the statistics will be 0 when first enabled, so only by enabling quota limits on empty subvolumes and then writing data can you ensure accurate statistics.

3.5 Commit interval

The resolution at which data are written to the file system is dictated by Btrfs itself and by system-wide settings. Btrfs defaults to a 30 seconds checkpoint interval in which new data are committed to the file system. This can be changed by appending the `commit` mount option in `/etc/fstab` for the Btrfs partition.

```
LABEL=arch64 / btrfs defaults,compress=zstd,commit=120 0 0
```

System-wide settings also affect commit intervals. They include the files under `/proc/sys/vm/*` and are out-of-scope of this wiki article. The kernel documentation for them is available at <https://docs.kernel.org/admin-guide/sysctl/vm.html> [\[10\]](#).

3.6 SSD TRIM

A Btrfs file system is able to free unused blocks from an SSD drive supporting the TRIM command. Asynchronous discard support is available with mount option `discard=async` , and is enabled by default as of **linux 6.2**. Freed extents are not discarded immediately, but grouped together and trimmed later by a separate worker thread, improving commit latency.

Asynchronous discard can safely be used alongside periodic trim [\[11\]](#) [\[10\]](#).

More information about enabling and using TRIM can be found in **Solid state drive#TRIM**.

4 Usage

4.1 Swap file

Note

There are some limitations of the implementation in Btrfs and Linux swap subsystem [\[12\]](#):

- Swap files on file systems spanning multiple devices are not supported
- Data **profile** must be `single`
- Snapshots cannot be created for subvolumes containing active swap files
- Swap files must be pre-allocated (i.e., no holes)
- Copy-on-write must be **disabled** individually for swap files (i.e., swap files must have the NOCOW attribute)

The last two limitations for swap files are automatically handled by the `btrfs filesystem mkswapfile` command.

For general information about swap files, see **Swap#Swap file**.

To properly create a swap file on Btrfs, first **create a subvolume** to store the swap file, as snapshots cannot be created for that subvolume afterwards. For example, create the `/swap` subvolume:

```
# btrfs subvolume create /swap
```

Tip

Consider **creating the subvolume** directly below the top-level subvolume, then **mount** it to `/swap` (or any other accessible location).

Create a 4 GiB swap file `swapfile` in `/swap/`:

```
# btrfs filesystem mkswapfile --size 4g --uuid clear /swap/swapfile
```

The `--size` option specifies the swap file size, with a default of 2 GiB and minimum of 40 KiB.

Activate the swap file:

```
# swapon /swap/swapfile
```

Finally, edit the **fstab** configuration to add an entry for the swap file:

```
/etc/fstab
```

```
/swap/swapfile none swap defaults 0 0
```

For additional information, see **fstab#Usage**.

For more information about removing swap files and other details, see **Swap#Swap file**.

Tip

To use hibernation functionality, in addition to configuring the swap file, you need to complete additional steps described in **Power management/Suspend and hibernate**.

4.2 Displaying used/free space

General linux userspace tools such as **df(1)** will inaccurately report free space on a Btrfs partition as they do not account for both file and metadata usage. It is recommended to use `btrfs filesystem usage` to query Btrfs partitions. For example, for a full breakdown of device allocation and usage stats:

```
# btrfs filesystem usage /
```

Note

`btrfs filesystem usage` and `btrfs filesystem df` show inaccurate results if RAID5 or RAID6 **profile** coexist with other profiles in the same filesystem (e.g. metadata using DUP while data uses RAID5) [\[13\]](#).

Alternatively, `btrfs filesystem df` allows a quick check on usage of allocated space without the requirement to run as root:

```
$ btrfs filesystem df /
```

See [\[14\]](#) for more information.

The same limitations apply to tools which analyze space usage for some subset of the file system, such as **du(1)** or **ncdu(1)**, as they do not consider **#Copy-on-Write (CoW)** or transparent **#Compression**. Instead, see **btdu^{AUR}** and **compsize** for Btrfs-aware alternatives.

4.3 Defragmentation

Btrfs supports online defragmentation through the mount option `autodefrag` ; see **btrfs(5) § MOUNT OPTIONS**. To manually defragment your root (not descending to subvolumes, mount points and directory symlinks), use:

```
# btrfs filesystem defragment -r /
```

Warning

Defragmenting will break up the reflinks of **#Copy-on-Write (CoW)** data (for example files copied with `cp` , **#Snapshots** or **deduplicated** data). This may cause considerable increase of space usage depending on

the broken up reflinks [15] [↗](#).

Note

Directory arguments without `-r` do not defragment files recursively but will defragment certain internal trees (extent tree and the subvolume tree). This has been confusing and could be removed in the future.

Tip

You can specify compression at the same time using options `-c` and `-L` ; see **#For existing files**.

4.4 RAID

Btrfs offers native "RAID" for **#Multi-device file systems**. Notable features which set Btrfs RAID apart from **mdadm** are self-healing redundant arrays and online balancing. See **the Btrfs wiki page** [↗](#) for more information. The Btrfs sysadmin page also **has a section** [↗](#) with some more technical background.

Warning

Parity RAID (RAID 5/6) code has multiple serious data-loss bugs in it. See the Btrfs Wiki's **RAID5/6 page** [↗](#) and a bug report on **linux-btrfs mailing list** [↗](#) for more detailed information. In June 2020, somebody posted a **comprehensive list of current issues** [↗](#) and a **helpful recovery guide** [↗](#).

4.4.1 Scrub



This article or section needs expansion.

Reason: Does Btrfs scrub examine subvolume or the device that subvolume resides on? [↗](#) (Discuss in **Talk:Btrfs**)

The **Btrfs Wiki Glossary** [↗](#) says that Btrfs scrub is "[a]n online file system checking tool. Reads all the data and metadata on the file system and uses checksums and the duplicate copies from RAID storage to identify and repair any corrupt data."

Note

A running scrub process will prevent the system from suspending, see **this thread** [↗](#) for details.

4.4.1.1 Start manually

To start a (background) scrub on the file system which contains `/` :

```
# btrfs scrub start /
```

To check the status of a running scrub:

```
# btrfs scrub status /
```

4.4.1.2 Start with a service or timer

The **btrfs-progs** package brings the `btrfs-scrub@.timer` unit for monthly scrubbing the specified mountpoint. **Enable** the timer with an escaped path, e.g. `btrfs-scrub@-.timer` for `/` and `btrfs-scrub@home.timer` for `/home`. You can use `systemd-escape -p /path/to/mountpoint` to escape the path; see **systemd-escape(1)** for details.

You can also run the scrub by **starting** `btrfs-scrub@.service` (with the same encoded path). The advantage of this over `btrfs scrub` (as the root user) is that the results of the scrub will be logged in the **systemd journal**.

On large NVMe drives with insufficient cooling (e.g. in a laptop), scrubbing can read the drive fast enough and long enough to get it very hot. If you are running scrubs with `systemd`, you can easily limit the rate of scrubbing with the `IOReadBandwidthMax` option described in **systemd.resource-control(5)** by using a **drop-in file**.

4.4.2 Balance

"A balance passes all data in the file system through the allocator again. It is primarily intended to rebalance the data in the file system across the devices when a device is added or removed. A balance will regenerate missing copies for the redundant RAID levels, if a device has failed." [16] [↗](#) See **Upstream FAQ page** [↗](#).

On a single-device filesystem, a balance may be also useful for (temporarily) reducing the amount of allocated but unused (meta)data chunks. Sometimes this is needed for fixing **"file system full" issues** [↗](#).

```
# btrfs balance start --bg /
# btrfs balance status /
```

4.5 Snapshots

"A snapshot is simply a subvolume that shares its data (and metadata) with some other subvolume, using Btrfs's COW capabilities." See **Btrfs Wiki SysadminGuide#Snapshots** [↗](#) and **the Btrfs documentation** [↗](#) for details.

To create a snapshot:

```
# btrfs subvolume snapshot source [dest/]name
```

To create a readonly snapshot, add the `-r` flag. To create a writable version of a readonly snapshot, simply create a snapshot of it.

Note

- It is possible for a snapshot to be converted in-place from readonly to writeable with `btrfs property set -f -ts '/path/to/snapshot' ro false`. However, this is not recommended because it **causes issues** [↗](#) with any future incremental send/receive. Making a new writeable snapshot prevents such issues.
- Snapshots are not recursive. Every nested subvolume will be an empty directory inside the snapshot.

4.6 Send/receive

A subvolume can be sent to stdout or a file using the `send` command. This is usually most useful when piped to a `Btrfs receive` command. For example, to send a snapshot named `/root_backup` (perhaps of a snapshot you made of `/` earlier) to `/backup`, you would do the following:

```
# btrfs send /root_backup | btrfs receive /backup
```

The snapshot that is sent *must* be readonly. The above command is useful for copying a subvolume to an external device (e.g. a USB disk mounted at `/backup` above).

The subvolume will be created on the receiving end. It does not need to be created manually.

Another example which creates: `/mnt/arch-v2/subvolumes/@var` :

```
# btrfs send --proto 2 --compressed-data '/mnt/arch/snapshots/@var' | btrfs receive '/mnt/arch-v2/subvolumes/'
```

The parameters `--proto 2` and `--compressed-data` used in the example might be useful for more efficient sending (assumes compressed data).

You can also send only the difference between two snapshots. For example, if you have already sent a copy of `root_backup` above and have made a new readonly snapshot on your system named `root_backup_new`, then to send only the incremental difference to `/backup`, do:

```
# btrfs send -p /root_backup /root_backup_new | btrfs receive /backup
```

Now, a new subvolume named `root_backup_new` will be present in `/backup`.

See [Btrfs Wiki's Incremental Backup page](#) and [#Incremental backup to external drive](#) on how to use this for incremental backups and for tools that automate the process.

4.7 Deduplication

Using copy-on-write, Btrfs is able to copy files or whole subvolumes without actually copying the data. However, whenever a file is altered, a new *proper* copy is created. Deduplication takes this a step further by actively identifying blocks of data which share common sequences and combining them into an extent with the same copy-on-write semantics.

Tools dedicated to deduplicate a Btrfs formatted partition include **duperemove** and **bees**. One may also want to merely deduplicate data on a file based level instead using e.g. **rmlint-git**^{AUR}, **rdfind**, **jdupes**^{AUR} or **dduper-git**^{AUR}. For an overview of available features of those programs and additional information, have a look at the [upstream Wiki entry](#).

4.8 Resizing

Warning

To avoid data loss, ensure that you back up your data before you begin any resizing task.

You can grow a file system to the maximum space available on the device, or specify an exact size. Ensure that you grow the size of the device or logical volume before you attempt to increase the size of the file system. When specifying an exact size for the file system on a device, either increasing or decreasing, ensure that the new size satisfies the following conditions:

- The new size must be greater than the size of the existing data; otherwise, data loss occurs.
- The new size must be equal to or less than the current device size because the file system size cannot extend beyond the space available.

Note

If you plan to also decrease the size of the logical volume that holds the file system, ensure that you decrease the size of the file system before you attempt to decrease the size of the device or logical volume.

To extend the file system size to the maximum available size of the device:

```
# btrfs filesystem resize max /
```

To extend the file system to a specific size:

```
# btrfs filesystem resize size /
```

Replace *size* with the desired size in bytes. You can also specify units on the value, such as K (kibibytes), M (mebibytes), or G (gibibytes). Alternatively, you can specify an increase or decrease to the current size by prefixing the value with a plus (+) or a minus (-) sign, respectively:

```
# btrfs filesystem resize +size /  
# btrfs filesystem resize -size /
```

5 Known issues

A few limitations should be known before trying.

5.1 Encryption

Btrfs has no built-in encryption support, but there is an ongoing effort [\[17\]](#) to incorporate encryption based on **Fscrypt**.

Users can however encrypt the partition before running `mkfs.btrfs`, see [dm-crypt/Encrypting an entire system](#). Another approach is **stacked filesystem encryption**.

5.2 btrfs check issues

The tool `btrfs check` has known issues and should not be run without further reading; see section [#btrfs check](#).

6 Tips and tricks

6.1 Partitionless Btrfs disk

Warning

This type of setup is not ideal for a boot device and it is recommended to instead set up a Btrfs partition along with a separate **EFI system partition**. Furthermore, GRUB strongly discourages installation to a partitionless disk.

Btrfs can occupy an entire data storage device, replacing the **MBR** or **GPT** partitioning schemes, using **subvolumes** to simulate partitions. However, using a partitionless setup is not required to simply **create a Btrfs file system** on an existing **partition** that was created using another method. There are some limitations to partitionless single disk setups:

- Cannot place other **file systems** on another partition on the same disk.
- Due to the previous point, having an **ESP** on this disk is not possible. Another device is necessary for **UEFI** boot.

To create a partitionless Btrfs disk, run the following command:

```
# mkfs.btrfs /dev/sdX
```

For example, use `/dev/sda` rather than `/dev/sda1`. The latter would format an existing partition instead of replacing the entire partitioning scheme. Because the root partition is Btrfs, make sure `btrfs` is compiled into the kernel, or put `btrfs` into `mkinitcpio.conf#MODULES` and **regenerate the initramfs**.

Install the **boot loader** like you would for a data storage device with a **Master Boot Record**. See [Syslinux#Manually](#) or [GRUB/Tips and tricks#Install to partition or partitionless disk](#). If your kernel does not boot due to `Failed to mount /sysroot.`, please add `GRUB_PRELOAD_MODULES="btrfs"` in `/etc/default/grub` and **generate** the grub configuration.

6.2 Ext3/4 to Btrfs conversion

Warning

There are many reports on the btrfs mailing list about incomplete/corrupt/broken conversions. Make sure you have *working* backups of any data you cannot afford to lose. See the [Convert](#)  page on the Btrfs wiki for more information.

Boot from an install CD, then convert by doing:

```
# btrfs-convert /dev/partition
```

Mount the partition and test the conversion by checking the files. Be sure to change the `/etc/fstab` to reflect the change (*type* to `btrfs` and *fs_passno* –the last field– to `0` as Btrfs does not do a file system check on boot). Also note that the UUID of the partition will have changed, so update `fstab` accordingly when using UUIDs. `chroot` into the system and rebuild your boot loader's menu list (see **Install from existing Linux**). If converting a root file system, while still `chrooted`, **regenerate the initramfs** or the system will not successfully boot.

Note

- If there is anything wrong, either unable to mount or write files to the newly converted Btrfs; there is always the option to rollback as long as the backup subvolume `/ext2_saved` is still there. Use the `btrfs-convert -r /dev/partition` command to rollback; this will discard any modifications to the newly converted Btrfs file system.
- If mounting a converted ext partition results in an error like
ERROR: dev extent devid 1 physical offset XXX len XXX is beyond device boundary XXX ,
the conversion might succeed if you create some empty space after the partition first. Rollback the conversion, then use `resize2fs` to shrink the partition by 50000 blocks and then grow it by 5000 blocks (creating 45000 free blocks). Then try the conversion again. See **upstream bug report** [🔗](#).

After confirming that there are no problems, complete the conversion by deleting the backup `ext2_saved` sub-volume. Note that you cannot revert back to ext3/4 without it.

```
# btrfs subvolume delete /ext2_saved
```

Finally, **balance** the file system to reclaim the space.

Remember that some applications which were installed prior have to be adapted to Btrfs.

Note

Ext3/4 to Btrfs conversion is a time consuming operation. For a 4 TiB file system and a regular HDD, it can take up to 10 hours.

6.3 Corruption recovery

Warning

The tool `btrfs check` has known issues, see section **#btrfs check**

btrfs-check cannot be used on a mounted file system. To be able to use *btrfs-check* without booting from a live USB, add it to the initial ramdisk:

```
/etc/mkinitcpio.conf
```

```
-----  
BINARIES=(btrfs)
```

Regenerate the initramfs.

Then if there is a problem booting, the utility is available for repair.

Note

If the fsck process has to invalidate the space cache (and/or other caches?), it is normal for a subsequent boot to hang up for a while (it may give console messages about btrfs-transaction being hung). The system should recover from this after a while.

See **btrfs-check(8)** for more information.

6.4 Booting into snapshots

In order to boot into a snapshot, the same procedure applies as for mounting a subvolume as your root partition, as given in section **mounting a subvolume as your root partition**, because snapshots can be mounted like subvolumes.

- If using **GRUB**, you can automatically populate your boot menu with Btrfs snapshots when regenerating the configuration file with the help of **grub-btrfs** or **grub-btrfs-git**^{AUR}.
- If using **refind**, you can automatically populate your boot menu with Btrfs snapshots with the help of **refind-btrfs**^{AUR}, after **enabling** `refind-btrfs.service`.
- If using **Limine**, you can install **limine-snapper-sync**^{AUR}, which automatically generates snapshot entries in your boot menu whenever your **Snapper** list changes after enabling `limine-snapper-sync.service`. See **Limine#Snapper snapshot integration for Btrfs** for more information.

6.5 Use Btrfs subvolumes with systemd-nspawn

See the **systemd-nspawn#Use Btrfs subvolume as container root** and **systemd-nspawn#Use temporary Btrfs snapshot of container** articles.

6.6 Reducing access time metadata updates

Because of the copy-on-write nature of Btrfs, simply accessing files can trigger the metadata copy and writing. Reducing the frequency of access time updates may eliminate this unexpected disk usage and increase performance. See **fstab#atime options** for the available options.

6.7 Incremental backup to external drive

The following packages use **btrfs send** and **btrfs receive** to send backups incrementally to an external drive. Refer to their documentation to see differences in implementation, features, and requirements.

- **snbk** — A command-line tool provided by **snapper** for copying snapper snapshots to remote or local backups.
<http://snapper.io/>  || **snapper**
- **btrbk** — Tool for creating snapshots and remote backups of Btrfs subvolumes.
<https://github.com/digint/btrbk>  || **btrbk**
- **snap-sync** — Use **Snapper** snapshots to backup to external drive or remote machine.
<https://github.com/wesbarnett/snap-sync>  || **snap-sync**^{AUR}

The following package allows backing up snapper snapshots to non-Btrfs file systems:

- **snapborg** — borgmatic-like tool which integrates snapper snapshots with **borg** backups.

<https://github.com/enzinger/snapborg>  || **snapborg**^{AUR}

6.8 Automatic snapshots

For the managing and automatic creation of Btrfs snapshot, one may use a snapshot manager such as **Snapper**, **Timeshift** or **Yabsnap**.

6.9 Automatic notifications

Desktop notifications help you notice serious Btrfs issues immediately, offering better awareness compared to having no notifications at all.

btrfs-desktop-notification^{AUR} provides desktop notifications for the following events:

- Booting into any read-only snapshot or system.
- Btrfs warning, error, or fatal messages appearing in the dmesg log.

See <https://gitlab.com/Zesko/btrfs-desktop-notification>  for more information and configuration.

7 Troubleshooting

See the **Btrfs Troubleshooting pages**  and **Btrfs Problem FAQ**  for general troubleshooting.

7.1 GRUB

7.1.1 Partition offset

The offset problem may happen when you try to embed `core.img` into a partitioned disk. It means that **it is OK** to embed GRUB's `core.img` into a Btrfs pool on a partitionless disk (e.g. `/dev/sdX`) directly.

GRUB can boot Btrfs partitions, however the module may be larger than other **file systems**. And the `core.img` file made by `grub-install` may not fit in the first 63 sectors (31.5KiB) of the drive between the MBR and the first partition. Up-to-date partitioning tools such as `fdisk` and `gdisk` avoid this issue by offsetting the first partition by roughly 1MiB or 2MiB.

7.1.2 Missing root



The factual accuracy of this article or section is disputed.

Reason: Suggests editing a non-configuration file manually. (Discuss in **Talk:Btrfs#Should not suggest to edit files in /usr/share**)

Users experiencing the following: `error no such device: root` when booting from a RAID style setup then edit `/usr/share/grub/grub-mkconfig_lib` and remove both quotes from the line `echo " search --no-floppy --fs-uuid --set=root ${hints} ${fs_uuid}"`. Regenerate the config for grub and the system should boot without an error.

Warning

Overwriting package files is discouraged and the file will be overwritten by next update.

7.2 Mounting timed out

Sometimes, especially with large RAID1 arrays, mounting might time out during boot with a journal message such as:

```
Jan 25 18:05:12 host systemd[1]: storage.mount: Mounting timed out. Terminating.
Jan 25 18:05:46 host systemd[1]: storage.mount: Mount process exited, code=killed, status=15/TERM
Jan 25 18:05:46 host systemd[1]: storage.mount: Failed with result 'timeout'.
Jan 25 18:05:46 host systemd[1]: Failed to mount /data.
Jan 25 18:05:46 host systemd[1]: Startup finished in 32.943s (firmware) + 3.097s (loader) + 7.247s (kernel)>
Jan 25 18:05:46 host kernel: BTRFS error (device sda): open_ctree failed
```

This can easily be worked around by providing a longer timeout via the systemd-specific mount option `x-systemd.mount-timeout` in **`fstab`**. For example:

```
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /data btrfs rw,relatime,x-systemd.mount-timeout=5min 0 0
```

7.3 BTRFS: open_ctree failed



The factual accuracy of this article or section is disputed.

Reason: The status is old. Replacing udev hook with systemd makes no sense since they both use systemd-udevd. (Discuss in **Talk:Btrfs**)

As of November 2014, there seems to be a bug in **`systemd`** or **`mkinitcpio`** causing the following error on systems with multi-device Btrfs file system using the `btrfs` hook in `mkinitcpio.conf`:

```
BTRFS: open_ctree failed
mount: wrong fs type, bad option, bad superblock on /dev/sdb2, missing codepage or helper program, or other error
```

In some cases, useful info is found in `syslog` - try `dmesg|tail` or so.

You are now being dropped into an emergency shell.

A workaround is to remove `btrfs` from the `HOOKS` array in `/etc/mkinitcpio.conf` and instead add `btrfs` to the `MODULES` array. Then **regenerate the `initramfs`** and reboot.

You will get the same error if you try to mount a raid array without one of the devices. In that case, you must add the `degraded` mount option to `/etc/fstab`. If your root resides on the array, you must also add

`rootflags=degraded` to your **kernel parameters**.

As of August 2016, a potential workaround for this bug is to mount the array by a single drive only in `/etc/fstab`, and allow Btrfs to discover and append the other drives automatically. Group-based identifiers such as UUID and LABEL appear to contribute to the failure. For example, a two-device RAID1 array consisting of 'disk1' and disk2' will have a UUID allocated to it, but instead of using the UUID, use only `/dev/mapper/disk1` in `/etc/fstab`. For a more detailed explanation, see the following **blog post** [↗](#).

Another possible workaround is to remove the `udev` hook in **mkinitcpio.conf** and replace it with the `systemd` hook. In this case, `btrfs` should *not* be in the `HOOKS` or `MODULES` arrays.

See the **original forums thread** [↗](#) and **FS#42884** [↗](#) for further information and discussion.

7.4 btrfs check



This article or section is out of date.

Reason: The "heavy development" status is old. (Discuss in **Talk:Btrfs**)

Warning

Since Btrfs is under heavy development, especially the `btrfs check` command, it is highly recommended to create a **backup** and consult **btrfs-check(8)** before executing `btrfs check` with the `--repair` switch.

The **btrfs-check(8)** command can be used to check or repair an unmounted Btrfs file system. However, this repair tool is still immature and not able to repair certain file system errors even those that do not render the file system unmountable.

7.5 Constant drive activity

Since the **kernel** version 6.2, `discard=async` **mount(8)** option is set by default. This **has been reported** [↗](#) to cause constant drive activity on some drives even while idle, as the discard queue is filled faster than it is processed. This can cause increased power usage, especially on NVMe-based drives.

As of kernel version 6.3, the default discard `iops_limit` has been changed from 100 to 1000 to address this issue. You can manually set it to a desired value on an old kernel version, e.g.

```
# echo 1000 > /sys/fs/btrfs/uuid/discard/iops_limit
```

Where `uuid` is the UUID of the Btrfs file system. The limit of `1000` will need to be tuned experimentally.

To set the parameter on boot, **systemd-tmpfiles** may be used, e.g. by creating the following file:

```
/etc/tmpfiles.d/btrfs-discard.conf
```

```
w /sys/fs/btrfs/uuid/discard/iops_limit - - - - 1000
```

Alternatively, one can disable asynchronous discard altogether by mounting using the `nodiscard` mount option in **fstab**, and instead use **Periodic TRIM**.

7.6 Device `total_bytes` should be at most X but found Y

If a drive has been moved from another computer or the order of devices has changed, and the difference in sizes reported is tiny (at most megabytes), it is possible **HPA (Host Protected Area)** [↗](#) is in effect.

To verify if HPA is enabled, use **hdparm**:

```
# hdparm -N DEVICE
```

The output provides two numbers: the number of visible sectors and the actual number of sectors. If they differ, HPA is enabled.

If motherboard sets this forcefully and the firmware provides no option to turn that off, the only option is to shrink the affected file system. This is most easily done on the original computer, or on any machine that does not apply HPA.

7.7 No space left on device

The blog post **Fixing Btrfs Filesystem Full Problems** [↗](#) suggests and explains the following checks/steps:

1. Clear space now (remove historical snapshots)
2. Is your filesystem really full? Mis-balanced metadata and/or data chunks (run `btrfs balance`)
3. Is your filesystem really full? Mis-balanced data chunks
4. Is your filesystem really full? Mis-balanced metadata
5. Balance cannot run because the filesystem is full (temporarily add a device such as USB key or loop device to the btrfs filesystem using `btrfs device add` before running `btrfs balance`)

For more up-to-date instructions regarding ENOSPC ("Out of disk space"), see **ENOSPC - No available disk space | Forza's Ramblings** [↗](#).

8 See also

- **Official site**
 - **Btrfs Documentation** [↗](#)
 - **Archived Btrfs Wiki** [↗](#)
- **Performance related**
 - **Btrfs on raw disks?** [↗](#)
 - **Varying leafsize and nodesize in Btrfs** [↗](#)
 - **Btrfs support for efficient SSD operation (data blocks alignment)** [↗](#)
 - **Is Btrfs optimized for SSDs?** [↗](#)
 - **Lzo vs. zLib** [↗](#)
- **Miscellaneous**
 - **Debian:Btrfs** [↗](#)

- **Gentoo:Btrfs** [↗](#)
- **Funtoo:BTRFS Fun** [↗](#)
- **Avi Miller presenting Btrfs** [↗](#) at SCALE 10x, January 2012.
- **Summary of Chris Mason's talk** [↗](#) from LFCS 2012
- **Btrfs: stop providing a bmap operation to avoid swapfile corruptions** [↗](#) 2009-01-21
- **Doing Fast Incremental Backups With Btrfs Send and Receive** [↗](#)

Category: File systems

This page was last edited on 25 October 2025, at 11:47.

Content is available under GNU Free Documentation License 1.3 or later unless otherwise noted.

[Privacy policy](#) [About ArchWiki](#) [Disclaimers](#) [Code of conduct](#) [Terms of service](#)